Arithmetic Operations in JavaScript

A quick-reference cheat sheet for everyday use

JavaScript does something interesting with arithmetic: it tries to be helpful by automatically converting strings to numbers when you use math operators. This is called **type coercion**, and it catches a lot of people off guard. The **+** operator is especially tricky because it doubles as both an addition operator and a string concatenation operator. Let's walk through each one.

1. Addition (+) -- the tricky one

Here's the deal: if either side of + is a string, JavaScript concatenates instead of adding. This is the only operator that behaves this way, which is why it trips people up the most.

Expression	Output	Why?
`1 + "1"`	`"11"`	Number meets string = concatenation
`"1" + 1`	`"11"`	String meets number = concatenation
`1 + 1`	`2`	Both numbers = actual math
`"1" + "1"`	`"11"`	Both strings = concatenation

Quick rule: If you see a + and one side is a string, the result will always be a string. Every other math operator converts strings to numbers automatically.

2. Subtraction (-)

Unlike addition, subtraction always tries to do math. If it gets a string, it converts it to a number first. If the string can't become a valid number, you get NaN (Not a Number).

Expression	Output	Why?
`1 - "1"`	`0`	Converts "1" to number 1
`"5" - 2`	`3`	Converts "5" to number 5
`"5" - "2"`	`3`	Converts both strings to numbers
`5 - "hello"`	`NaN`	"hello" is not a valid number

3. Multiplication (*)

Same story here -- strings get converted to numbers automatically.

Expression	Output	Why?
`2 * "3"`	6	Converts "3" to number 3
`"2" * 3`	6	Converts "2" to number 2
`"2" * "3"`	6	Both strings become numbers

4. Division (/)

Expression	Output	Why?
`6 / "2"`	3	Converts "2" to number 2
`"6" / 2`	3	Converts "6" to number 6
`"6" / "2"`	3	Both strings become numbers

5. Modulus (%)

The modulus operator gives you the remainder after division. It also converts strings to numbers.

Expression	Output	Why?
`5 % "2"`	1	Remainder of 5 / 2
`"5" % 2`	1	Same thing, string converted
`"5" % "2"`	1	Both converted, same result

6. Exponentiation (**)

Raises the left side to the power of the right side. Works just like Math.pow() but with cleaner syntax.

Expression	Output	Why?
`2 ** "3"`	8	2 to the power of 3
`"2" ** 3`	8	String "2" becomes number 2

`"2" ** "3"`	8	Both converted automatically

7. Increment (++) and Decrement (--)

These operators add or subtract 1 from a value. The placement matters -- putting it before (prefix) or after (postfix) the variable changes when the operation happens.

```
// Prefix increment: converts to number FIRST, then adds 1
let a = "5";
console.log(++a);   // 6  (now a number)

// Postfix increment: returns current value, THEN increments
let b = "5";
console.log(b++);   // "5" (still the original string)
console.log(b);     // 6   (now incremented)

// Same logic applies to decrement (--)
let c = "5";
console.log(--c);   // 4  (prefix: convert + decrement first)
let d = "5";
console.log(d--);   // "5" (postfix: return first, then decrement)
console.log(d);     // 4  (now decremented)
```

8. Unary Plus (+) and Unary Negation (-)

These are shorthand ways to convert a string to a number. The unary plus just converts; the unary negation converts and flips the sign.

```
// Unary plus: quick way to convert string to number
console.log(+"42");      // 42
console.log(+"hello");   // NaN

// Unary negation: converts and negates
console.log(-"42");      // -42
console.log(-"hello");   // NaN
```

The big takeaway: Every arithmetic operator except + converts strings to numbers automatically. The + operator is the odd one out -- it prefers concatenation whenever a string is involved. When in doubt, use Number(), parseInt(), or the unary + to convert explicitly.

Template Literals in JavaScript

A cleaner, more readable way to work with strings

What are Template Literals?

Template literals are a way to create strings in JavaScript using backticks (`) instead of the usual single or double quotes. They were introduced in ES6 and they solve a bunch of annoyances that developers had been dealing with for years -- things like messy string concatenation, multi-line strings, and escaping quote characters.

```
let message = `This is a template literal`;
```

1. Multi-line Strings

With regular strings, you need escape characters or concatenation to write multi-line text. Template literals just let you hit Enter and keep typing. The line breaks are preserved as-is.

```
let message = `hi this is my js code
               im so happy
               this is my js learning`;

// No \n needed, no string concatenation -- it just works.
```

2. No Need for Escape Characters

Since template literals use backticks, single and double quotes can live happily inside without any escaping. This is one of those small things that saves you a lot of headaches.

```
// The old way -- escaping is annoying
let m1 = 'it\'s my code';
let p1 = "it\"s my code";

// The template literal way -- clean and simple
let mg = `hi it's my code`;
```

3. Variable Interpolation with ${}

This is the feature people love the most. Instead of breaking your string apart with + signs to insert variables, you just drop them right into the string using ${} placeholders.

```
let empname = 'Tom';
let age = 30;

// Old way (string concatenation)
console.log('hey ' + empname + ' and our age is ' + age);

// Template literal way (much cleaner)
console.log(`hey ${empname} and your age is ${age}`);
```

4. Expression Evaluation Inside ${}

You can put any valid JavaScript expression inside the ${} -- not just variables. Math, function calls, ternary operators -- it all works.

```
let a = 100;
let b = 200;

console.log(`the sum is: ${a + b}`);   // 300
console.log(`${a}, ${b}`);             // 100, 200
console.log(`is adult: ${age >= 18 ? "yes" : "no"}`);
```

When to Use Template Literals

You don't have to use template literals everywhere, but here are the situations where they really shine:

- When you need to insert variables into a string -- this is the primary use case
- For multi-line strings -- no more \n or awkward concatenation
- When your string contains both single and double quotes
- For building dynamic HTML or messages
- When doing calculations or calling functions inside a string

Things to remember: Template literals use backticks (`) -- that key is usually to the left of the 1 key on your keyboard. Variables and expressions go inside ${ }. They preserve whitespace and line breaks exactly as you type them. Supported in all modern browsers and Node.js.

Quick Comparison

Feature	Regular Strings	Template Literals
Syntax	"..." or '...'	`...`
Multi-line	Needs \n	Just press Enter
Variables	"hi " + name	`hi ${name}`
Quotes inside	Need escaping	No escaping needed
Expressions	Not possible inline	${a + b} works

That's it -- these two topics cover a good chunk of the everyday string and number quirks you'll run into in JavaScript. Keep this handy and refer back whenever something doesn't behave the way you expect.